Postgresql

Find out which table a given toast belongs to

Run this query

```
select n.nspname, c.relname
from pg_class c
inner join pg_namespace n on c.relnamespace = n.oid
where reltoastrelid = (
    select oid
    from pg_class
    where relname = 'pg_toast_16518'
    and relnamespace = (SELECT n2.oid FROM pg_namespace n2 WHERE n2.nspname = 'pg_toast') );
```

```
 nspname |       relname
---------+---------------------
 public  | merge_request_diffs
(1 row)
```

Get a list of all active queries

```
SELECT pid, state, age(clock_timestamp(), query_start) as duration, query
FROM pg_stat_activity
WHERE query != '<IDLE>' AND query NOT ILIKE '%pg_stat_activity%' AND state != 'idle'
ORDER BY age(clock_timestamp(), query_start) DESC;
```

Get a list of slow queries (more than 1 second)

```
SELECT pid, state, age(clock_timestamp(), query_start) as duration, query
FROM pg_stat_activity
WHERE query != '<IDLE>' AND query NOT ILIKE '%pg_stat_activity%' AND state != 'idle' AND age
ORDER BY age(clock_timestamp(), query_start) DESC;
```

Get a list of queries that are waiting

```
SELECT pid, query, age(clock_timestamp(), query_start) AS query_duration
FROM pg_catalog.pg_stat_activity WHERE  waiting
ORDER BY age(clock_timestamp(), query_start) DESC;
```

Get a list of locked queries with the query that is blocking it

```
SELECT blockingl.relation::regclass,
  blockeda.pid AS blocked_pid, blockeda.query as blocked_query,
  blockedl.mode as blocked_mode,
  age(clock_timestamp(), blockeda.query_start) as blocked_query_duration,
```

```
  blockinga.pid AS blocking_pid, blockinga.query as blocking_query,
  blockingl.mode as blocking_mode,
  age(clock_timestamp(), blockinga.query_start) as blocking_query_duration
FROM pg_catalog.pg_locks blockedl
JOIN pg_stat_activity blockeda ON blockedl.pid = blockeda.pid
JOIN pg_catalog.pg_locks blockingl ON(blockingl.relation=blockedl.relation
  AND blockingl.locktype=blockedl.locktype AND blockedl.pid != blockingl.pid)
JOIN pg_stat_activity blockinga ON blockingl.pid = blockinga.pid
WHERE NOT blockedl.granted;
```

Run pgbadger in the primary database server

- sudo up
- `pgbadger | /usr/bin/pgbadger -o output.txt -`

Triggering a Failover

See the PostgreSQL Switchover document for instructions

Setting up Secondaries

See the PostgreSQL Replica document for instructions

Rebuild a corrupt index

Summary: we must build a new index concurrently, so as not to contend with production traffic, and then replace the corrupt index with this new one.

Run these SQL commands in `gitlab-psql` shell on the **primary** database instance.

Find the size of the index:

```
select pg_size_pretty(pg_indexes_size('index_blah'));
```

As a very rough rule of thumb, we can expect index creation to take a few minutes per GB on production.

Set your statement timeout to be long enough to create the new index:

```
set statement_timeout to '1h';
```

Find the definition of the index:

```
select indexdef from pg_indexes where indexname = 'index_blah';
```

Create a replacement index with a different name, based on the definition about **but ensuring to use `CONCURRENTLY`**:

```
CREATE INDEX CONCURRENTLY index_blah_rebuild ON foo USING some_algo (bar, baz);
```

Rename the corrupt index, then name the new index to the corrupt index's old name, in a transaction:

```
BEGIN;
ALTER INDEX index_blah RENAME TO index_blah_old;
ALTER INDEX index_blah_rebuild RENAME TO index_blah;
END;
```

Verify that the new index is receiving traffic:

```
select * from pg_stat_user_indexes where indexrelname = 'index_blah';
```

`idx_tup_read` and/or `idx_tup_fetch` should increase over time.

Drop the old index:

```
DROP INDEX index_blah_old;
```

Indexes corruption can occur when string sorting order changes. String sorting order can change on glibc upgrade, or postgres upgrade. When upgrading postgres we must be careful to check for index corruption and rebuild indexes if necessary.